Section 9: Sorting

This section is part of the 5-hour LeetCode DSA Foundations course by Mukul Raina

- YouTube Course: https://youtu.be/YJMBv4Q2Tg8?si=Q1vDzusUJf753GG3
- YouTube Channel: https://www.youtube.com/@mukulraina
- Instructor LinkedIn: https://www.linkedin.com/in/mukulraina/

SLIDE 81: Section Intro, Sorting

SLIDE 82: The Sorting Landscape

SLIDE 82B: Merge Sort vs Quick Sort

SLIDE 83: Java Built-In Sorting and Custom Comparators

SLIDE 84: Sort-Then-Solve: The Strategy Pattern

SLIDE 85: Worked Example: Merge Intervals

SLIDE 86: Wrong vs Right: Sorting Mistakes

SLIDE 87: Interview Practice: Sorting

SLIDE 88: Section Cheat Sheet

SLIDE 81: Section Intro, Sorting

Section 9: Sorting

Sorting appears in interviews in two distinct ways. First, interviewers ask you to explain the trade-offs between sorting algorithms: why would you choose Merge Sort over Quick Sort, or when does Counting Sort outperform both? Second, and more commonly, sorting is a preprocessing step that unlocks simpler solutions to problems that are otherwise quadratic.

You have already seen this principle in action. In Section 6, Three Sum sorted the array first so two pointers could work. In Section 8, binary search required sorted input as a precondition. Many interval problems, scheduling problems, and duplicate-detection problems become simple linear scans after a sort.

What We'll Cover

1. The sorting algorithm landscape and when each matters
2. Merge Sort vs Quick Sort: the trade-off interviewers ask about
3. Java built-in sorting and custom comparators
4. Sort-then-solve as a strategy for breaking out of O(n^2) brute forces
5. Merge Intervals as the canonical sort-then-solve problem

Connection to Other Sections

Sorting is prerequisite knowledge for two pointers on unsorted input (Section 6), binary search (Section 8), and many greedy problems (Section 13) including interval scheduling. When you see a problem where sorted order would make the solution obvious, that is a sort-then-solve signal.

Mukul Raina | youtube.com/@mukulraina | linkedin.com/in/mukulraina

SLIDE 82: The Sorting Landscape

Not all sorting algorithms are equal. Each has different guarantees on time, space, stability, and worst-case behavior. Interviews rarely ask you to implement a sort from scratch, but they do ask you to explain trade-offs and choose the right one.

Algorithm Comparison

Algorithm	Best	Average	Worst	Space	Stable?	Key Property
Merge Sort	O(n log n)	O(n log n)	O(n log n)	O(n)	Yes	Guaranteed worst-case performance
Quick Sort	O(n log n)	O(n log n)	O(n^2)	O(log n)	No	In-place, fastest in practice on average
Heap Sort	O(n log n)	O(n log n)	O(n log n)	O(1)	No	In-place with guaranteed O(n log n)
Counting Sort	O(n + k)	O(n + k)	O(n + k)	O(k)	Yes	Breaks O(n log n) barrier for bounded values
Insertion Sort	O(n)	O(n^2)	O(n^2)	O(1)	Yes	Optimal for nearly-sorted or tiny inputs

k = range of values in Counting Sort. Stability means equal elements keep their original relative order.

Counting Sort: Breaking the O(n log n) Barrier

Comparison-based sorting has a proven lower bound of O(n log n). No comparison sort can do better in the worst case. Counting Sort bypasses this by not comparing elements at all. Instead, it counts occurrences and reconstructs the output. This works when the value range k is small relative to n (e.g., sorting exam scores 0-100, sorting characters a-z).

Mukul Raina | youtube.com/@mukulraina | linkedin.com/in/mukulraina

SLIDE 82B: Merge Sort vs Quick Sort

These are the two algorithms you need to understand at depth. Every other sort is either a special case or rarely asked.

Merge Sort

Divides the array in half, recursively sorts each half, then merges the two sorted halves. The merge step does the real work: walk two pointers through the two halves, always picking the smaller element.

Guaranteed O(n log n) in all cases. Stable (equal elements stay in original order). Costs O(n) extra space for the merge buffer. Used when worst-case guarantees or stability are required.

Quick Sort

Picks a pivot, partitions the array so everything smaller is on the left and everything larger is on the right, then recursively sorts both sides. The partition step does the real work.

O(n log n) on average, O(n^2) in the worst case (when the pivot is always the smallest or largest element). Not stable. In-place (O(log n) space for the recursion stack). Faster in practice than Merge Sort due to better cache locality and lower constant factors.

Detailed Trade-Off Comparison

Dimension	Merge Sort	Quick Sort
Worst-case time	O(n log n) always	O(n^2) with bad pivots
Average-case time	O(n log n)	O(n log n), faster constants
Space	O(n) extra	O(log n) recursion stack
Stability	Stable	Not stable
Cache performance	Poor (non-contiguous merge buffer)	Good (in-place, sequential access)
Parallelization	Easy (independent halves)	Harder (partition is sequential)
Used by Java	TimSort for objects (merge-based)	Dual-pivot QuickSort for primitives

When Interviewers Ask “Which Is Better?”

This is a trade-off question, not a trick question. Neither is universally better.

Choose Merge Sort when: you need guaranteed O(n log n) worst case, you need a stable sort, you are sorting linked lists (merge is natural, no random access needed), or you are sorting external data that does not fit in memory.

Choose Quick Sort when: you need in-place sorting with minimal extra memory, average-case performance matters more than worst-case guarantees, you are sorting arrays in memory where cache locality helps, or you want the fastest practical sort for general use.

Interview Tip: If asked “which sorting algorithm would you use?”, the strongest answer acknowledges both options and states the criteria for choosing. “I’d use Merge Sort if I need stability or worst-case guarantees, and Quick Sort for in-place sorting where average-case speed matters most. Java’s Arrays.sort uses a variant of each depending on the data type.”

Mukul Raina | youtube.com/@mukulraina | linkedin.com/in/mukulraina

---

SLIDE 83: Java Built-In Sorting and Custom Comparators

In interviews, use Arrays.sort() unless you are explicitly asked to implement a sorting algorithm. Reimplementing sort wastes time and introduces bugs.

Java’s Sorting Internals

Data Type	Algorithm Used	Stable?	Why
Primitive arrays (int[], double[])	Dual-Pivot Quick Sort	No	Speed: primitives have no identity, stability is meaningless
Object arrays (Integer[], String[])	TimSort (merge-based)	Yes	Stability matters: objects may have equal keys but different identity
Collections.sort(List)	TimSort	Yes	Delegates to array sort internally

TimSort is a hybrid of Merge Sort and Insertion Sort. It exploits existing order in the data (called "runs") and is O(n) on already-sorted input.

Custom Comparators

When default ordering is not what you need, pass a comparator to Arrays.sort().

```
Integer[] arr = {-3, 1, -2, 5, -4};
Arrays.sort(arr, (a, b) -> Integer.compare(Math.abs(a), Math.abs(b)));

String[] words = {"banana", "fig", "apple", "kiwi"};
Arrays.sort(words, (a, b) -> Integer.compare(a.length(), b.length()));

int[][] intervals = {{3,5}, {1,4}, {2,6}};
Arrays.sort(intervals, (a, b) -> Integer.compare(a[0], b[0]));
```

The lambda (a, b) → ... receives two elements. Return a negative value if a should come first, positive if b should come first, zero if equal.

The Integer Overflow Pitfall

```
Arrays.sort(arr, (a, b) -> a - b);
```

This looks correct but overflows when a and b are far apart. If a = Integer.MIN_VALUE and b = 1, then a - b overflows to a large positive number, producing the wrong order.

```
Arrays.sort(arr, (a, b) -> Integer.compare(a, b));
```

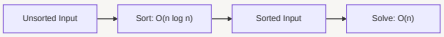
Integer.compare handles all values safely. Always use this form. The subtraction shortcut is a trap that works on most test cases but fails on edge cases, which is exactly the kind of bug interviewers look for.

Mukul Raina | youtube.com/@mukulraina | linkedin.com/in/mukulraina

---

SLIDE 84: Sort-Then-Solve: The Strategy Pattern

Many problems have an O(n^2) brute force where sorting the input first converts the quadratic step into a linear scan. The sort costs O(n log n) and the scan costs O(n), giving O(n log n) total. Since O(n log n) is usually within the acceptable complexity for n up to 10^6, this is a highly practical strategy.



Problems That Unlock After Sorting

Problem	Without Sorting	After Sorting	Why Sorting Helps
Merge Intervals	O(n^2) compare all pairs	O(n) linear merge	Overlapping intervals become adjacent
Three Sum	O(n^3) check all triples	O(n^2) fix-one + two-pointers	Two pointers require sorted order
Meeting Rooms (can attend all?)	O(n^2) check all pairs for overlap	O(n) check consecutive pairs	Conflicts only exist between adjacent intervals
Meeting Rooms II (min rooms)	Complex event tracking	O(n) with sorted start/end arrays	Sweep line on sorted events
Find Duplicates	O(n^2) nested loop	O(n) check adjacent elements	Duplicates become neighbors
Largest Number	No clear brute force	Custom comparator	“9” > “80” because “980” > “809”
Valid Anagram (sort approach)	N/A (HashMap is better)	O(n log n) compare sorted strings	Sorted anagrams are identical

When Sorting is NOT Worth It

Situation	Why Not	What to Use Instead
Need original indices	Sorting destroys position information	HashMap (Section 5)
O(n) solution exists without sorting	O(n) < O(n log n)	HashSet, frequency count, or linear scan
Input is already sorted	Sorting is redundant O(n log n)	Two pointers (Section 6) or binary search (Section 8)
Need stability and using primitive arrays	Arrays.sort on primitives is not stable	Box to Integer[] or use custom merge

The Sort-Then-Solve Decision Heuristic

When you are stuck on an O(n^2) brute force, ask yourself: “Would this problem be easier if the input were sorted?” If the answer is yes, sort first and see if a linear or two-pointer scan finishes the job.

Mukul Raina | youtube.com/@mukulraina | linkedin.com/in/mukulraina

---

SLIDE 85: Worked Example: Merge Intervals

Problem: Merge Intervals (LeetCode 56)

Given an array of intervals where intervals[i] = [start_i, end_i], merge all overlapping intervals and return an array of the non-overlapping intervals that cover all the intervals in the input.

Why Sorting First

Without sorting, you need to compare every interval with every other interval to check for overlaps. That is O(n^2). After sorting by start time, overlapping intervals are adjacent in the array. A single left-to-right scan merges them in O(n).

Full Java Solution

```java
public int[][] merge(int[][] intervals) {
    Arrays.sort(intervals, (a, b) -> Integer.compare(a[0], b[0]));
    List<int[]> result = new ArrayList<>();
    result.add(intervals[0]);
    for (int i = 1; i < intervals.length; i++) {
        int[] last = result.get(result.size() - 1);
        if (intervals[i][0] <= last[1]) {
            last[1] = Math.max(last[1], intervals[i][1]);
        } else {
            result.add(intervals[i]);
        }
    }
    return result.toArray(new int[result.size()][]);
}
```

Trace Through [[1,3], [2,6], [8,10], [15,18]]

```
After sorting by start (already sorted): [[1,3], [2,6], [8,10], [15,18]]

Start: result = [[1,3]]

i=1: [2,6]. Start 2 <= end 3 of last interval. Merge: [1, max(3,6)] = [1,6]
     result = [[1,6]]

i=2: [8,10]. Start 8 > end 6 of last interval. No overlap. Add new.
     result = [[1,6], [8,10]]

i=3: [15,18]. Start 15 > end 10 of last interval. No overlap. Add new.
     result = [[1,6], [8,10], [15,18]]
```

Complexity

Time: O(n log n) for the sort, O(n) for the scan. Total O(n log n).

Space: O(n) for the result list. The sort uses O(log n) for Java's internal recursion stack.

The Merge Condition

Two intervals overlap if and only if the start of the second is less than or equal to the end of the first (after sorting by start). When they overlap, the merged interval is [start of first, max(end of first, end of second)]. The max is necessary to handle fully nested intervals like [1,10] and [2,5], where the first interval's end is larger.

Mukul Raina | youtube.com/@mukulraina | linkedin.com/in/mukulraina

---

SLIDE 86: Wrong vs Right: Sorting Mistakes

Wrong: Implementing Bubble Sort or Selection Sort

Implementing O(n^2) sorts in an interview signals that you are not aware of O(n log n) alternatives. Unless the interviewer explicitly asks for a specific algorithm, use Arrays.sort() and spend your time on the actual problem.

Wrong: Stating O(n) for a Solution That Sorts First

If your solution starts with Arrays.sort(), the total complexity is at least O(n log n), not O(n). The sort dominates. A common mistake is analyzing only the scan after the sort and forgetting to include the sorting cost.

Wrong: Using a - b as a Comparator

This overflows for extreme values. At n = 200,000 with values near Integer.MIN_VALUE and Integer.MAX_VALUE, the subtraction produces wrong ordering and incorrect results. Use Integer.compare(a, b) unconditionally.

Wrong: Forgetting That Sorting Changes Indices

If the problem asks you to return original indices (like Two Sum), sorting destroys that information. You need a different approach, typically a HashMap that preserves the original index mapping.

Wrong vs Right Summary

Mistake	What Goes Wrong	Correct Approach
Implementing Bubble/Selection Sort	O(n^2), wastes interview time	Arrays.sort() by default
Stating O(n) after sorting	Ignores dominant sort cost	State total: O(n log n)
Comparator a - b	Integer overflow on extreme values	Integer.compare(a, b)
Sorting when indices are needed	Loses original positions	Use HashMap for index preservation
Calling Arrays.sort(int[]) and assuming stable	Primitive sort is NOT stable in Java	Use Integer[] or custom sort if stability matters
Not handling single-element or empty input	ArrayIndexOutOfBoundsException	Check length before accessing intervals[0]

Mukul Raina | youtube.com/@mukulraina | linkedin.com/in/mukulraina

---

SLIDE 87: Interview Practice: Sorting

Practice Problem

“Given a collection of intervals, merge all overlapping intervals.”

Expected Walkthrough Using the Five-Step Framework

Step 1: Understand (30 seconds)

“I’m given an array of [start, end] pairs and I need to combine any intervals that overlap. For example, [1,3] and [2,6] overlap, so they merge into [1,6]. I should ask: can intervals be unsorted? Yes. Can I modify the input? I’ll assume yes unless told otherwise.”

Step 2: Identify the Pattern (15 seconds)

“If I sort by start time, overlapping intervals will be adjacent. This is a sort-then-solve problem.”

Step 3: Plan (30 seconds)

“Sort by start time. Initialize result with the first interval. For each remaining interval, check if it overlaps with the last interval in the result. If yes, merge by extending the end. If no, add it as a new interval.”

Step 4: Implement (5-7 minutes)

Code the solution while narrating: “I’m sorting by start time using Integer.compare to avoid overflow. Then I iterate starting from index 1, comparing each interval’s start against the last merged interval’s end.”

Step 5: Verify (1-2 minutes)

“Let me trace through [[1,3],[2,6],[8,10],[15,18]]. After sorting, same order. First interval [1,3] goes in result. [2,6] overlaps because 2 <= 3, merge to [1,6]. [8,10] does not overlap because 8 > 6, add new. [15,18] does not overlap because 15 > 10, add new. Result: [[1,6],[8,10],[15,18]]. Correct.”

“Time is O(n log n) for the sort plus O(n) for the scan, so O(n log n) total. Space is O(n) for the result list.”

Common Fresher Mistakes

Mistake	Why It Fails
Trying to merge without sorting first	Have to compare every pair: O(n^2) and complex logic
Sorting by end time instead of start time	Adjacent intervals may not be overlapping pairs
Using max only for the end during merge	Not a mistake, but forgetting max causes failures on nested intervals like [1,10] and [2,3]
Not converting result list back to array	Return type is int[][], not List
Skipping the empty/single-interval edge case	Arrays.sort on empty array is fine, but accessing intervals[0] on empty input crashes

Mukul Raina | youtube.com/@mukulraina | linkedin.com/in/mukulraina

---

SLIDE 88: Section Cheat Sheet

Algorithm Properties Quick Reference

Algorithm	Time (Worst)	Space	Stable?	When to Choose
Merge Sort	O(n log n)	O(n)	Yes	Need guaranteed worst-case or stability
Quick Sort	O(n^2)	O(log n)	No	Need in-place, average-case speed
Heap Sort	O(n log n)	O(1)	No	Need in-place with worst-case guarantee
Counting Sort	O(n + k)	O(k)	Yes	Bounded value range, k is small
TimSort (Java objects)	O(n log n)	O(n)	Yes	Default for Arrays.sort on objects

Sort-Then-Solve Decision

Stuck on O(n^2)? Ask: “Would this be easier if the input were sorted?” If yes, sort first. Total becomes O(n log n), which is acceptable for n up to 10^6.

Do NOT sort when: you need original indices (use HashMap), an O(n) solution exists without sorting (use HashSet or frequency count), or the input is already sorted (use two pointers or binary search directly).

Custom Comparator Template

```
Arrays.sort(items, (a, b) -> Integer.compare(a.property, b.property));
```

Never use (a, b) → a - b. Always use Integer.compare to prevent overflow.

Key Facts

Fact	Detail
Comparison sort lower bound	O(n log n) worst case. No comparison sort can beat this.
Sort + linear scan total	O(n log n), not O(n). The sort dominates.
Arrays.sort(int[])	Dual-pivot Quick Sort. Not stable.
Arrays.sort(Integer[])	TimSort. Stable.
Sorting a linked list	Merge Sort is natural (no random access needed)
Counting Sort applicability	Only when value range k is small relative to n

Mukul Raina | youtube.com/@mukulraina | linkedin.com/in/mukulraina